Agenda

- Hierarchial Clustering
  - Agglomerative Clustering
  - Divise Clustering
- Proximity matrix & Dendogram
- Code

Problem Statement

→ Data Scientist at Amazon

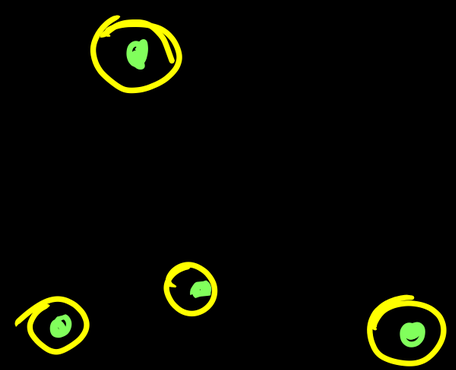
Hierarchial Clustering

↳ Agglomerative ✓ ⟶ Bottom-up

↳ Divisive (Not used)

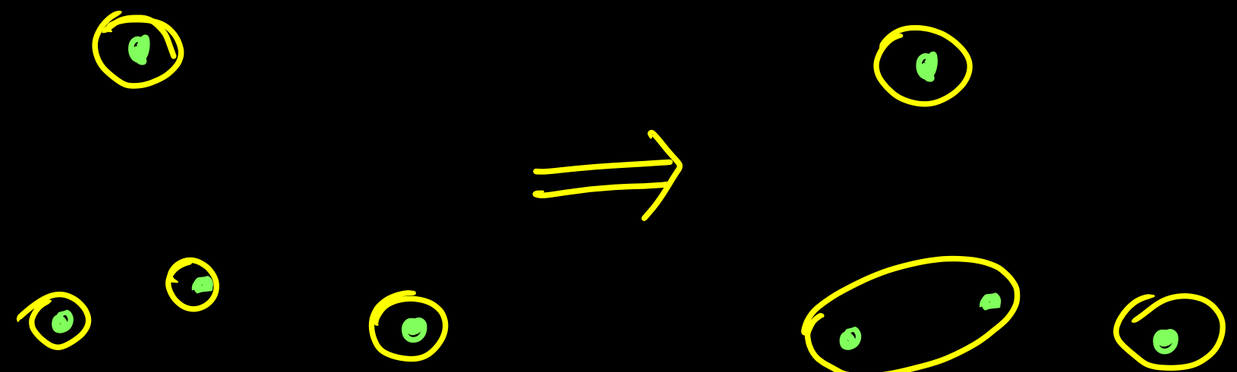
↳ top-bottom

4 cluster



4 cluster

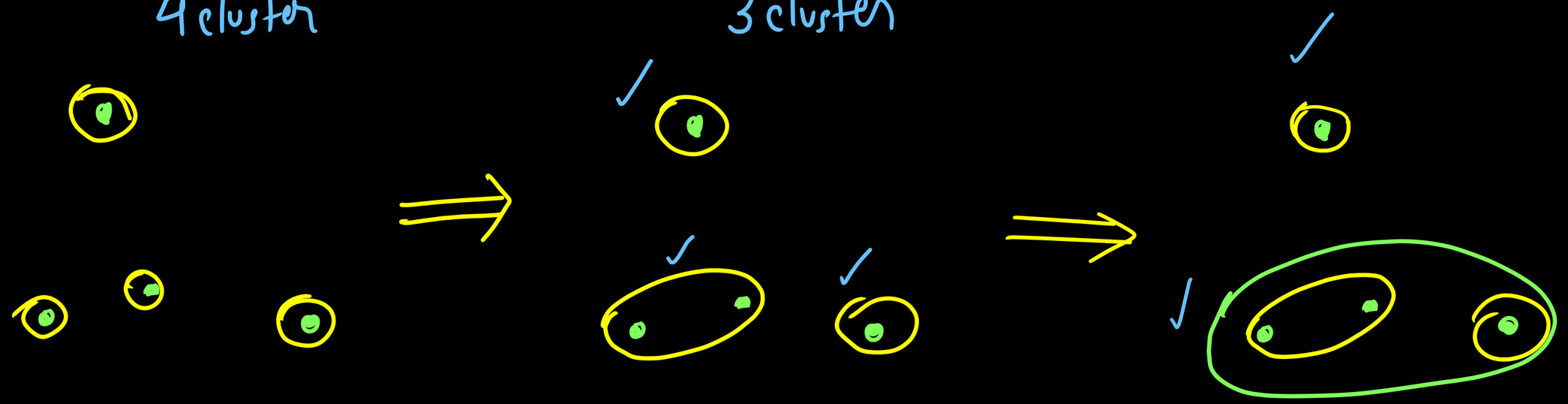
3 cluster



1 cluster

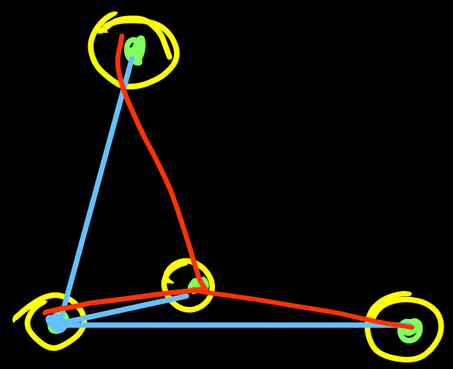
4 cluster

3 cluster

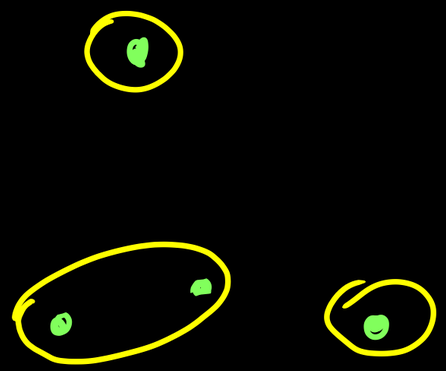


1 cluster

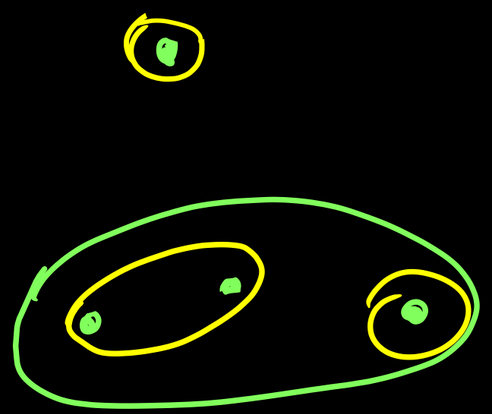
4 cluster



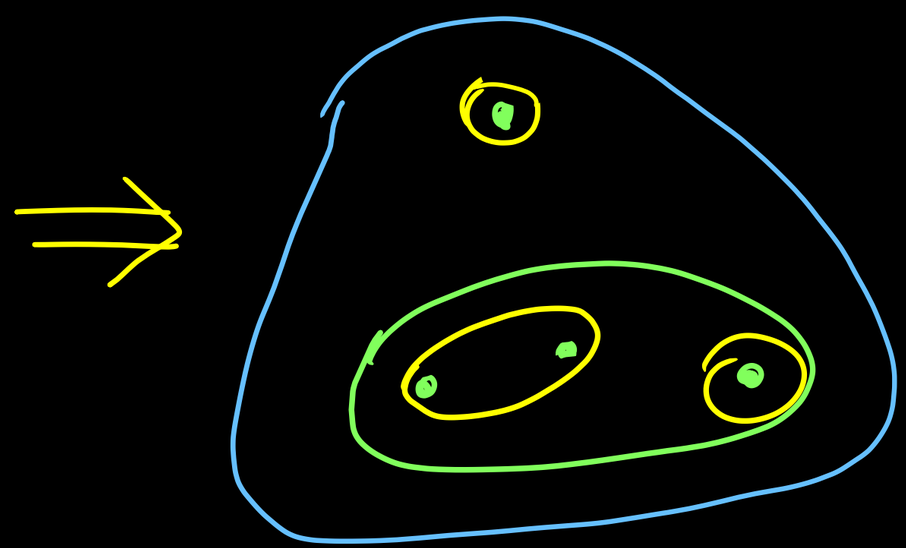
3 cluster



2 cluster



1 cluster



Agglomerative Clustering

→ Each point as a cluster

→ Repeat until tot.cluster = 1 :
{

→ Merge the nearest cluster

}

Algorithm

→ Start with n-data point, each point as a single cluster

→ Repeat until tot. cluster = 1

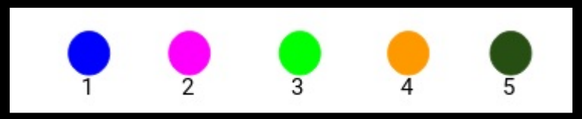
→ Calculate the pairwise dist. using Proximity matrix

→ Merge cluster by identifying closest cluster

→ Update the Proximity matrix

Student_ID	Marks
1	10
2	7
3	28
4	20
5	35

n



max (10, 7)
= 10

Student_ID	Marks
1	10
2	7
3	28
4	20
5	35



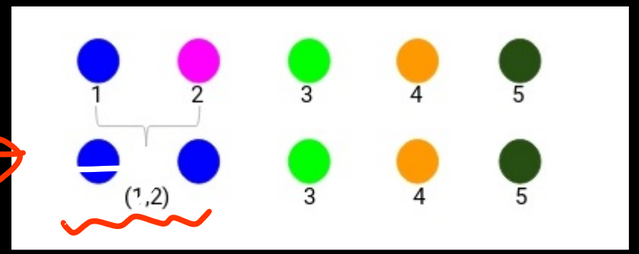
Proximity Matrix = Matrix of distance

ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0

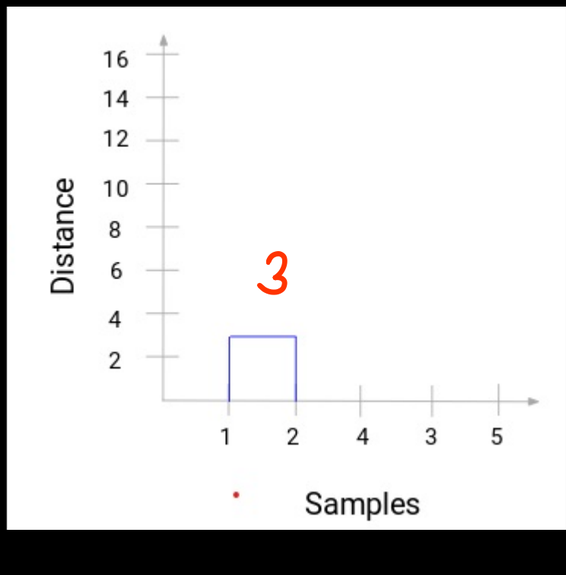
n x n

→ Dendogram = Tree like diagram that records sequence of merges.

↳ Height of join = Distance b/w cluster



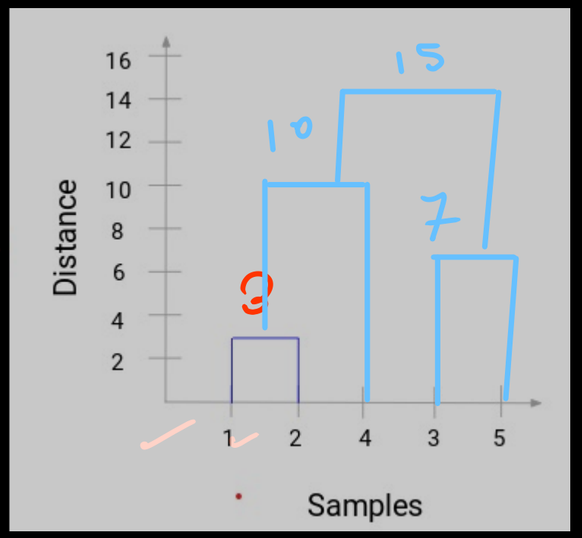
ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0



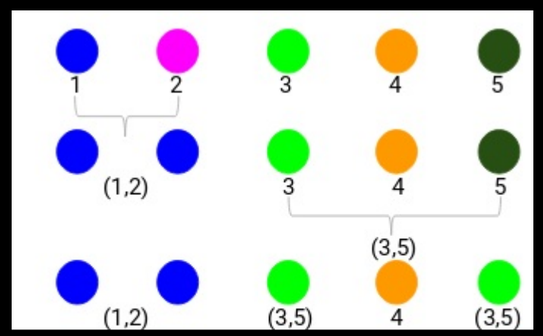
Student_ID	Marks
(1,2)	10
3	28
4	20
5	35

Student_ID	Marks
(1,2)	10
3	28
4	20
5	35

ID	(1,2)	3	4	5
(1,2)	0	18	10	25
3	18	0	8	7
4	10	8	0	15
5	25	7	15	0



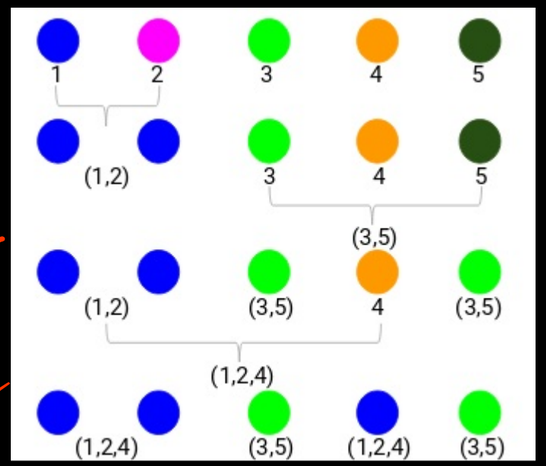
max (28, 35) = 35



Student ID	Marks
(1,2)	10
(3,5)	35
4	20

max(20, 35)

	(1,2)	(3,)	4
(1,2)	0	25	10
(3,5)	25	0	15
4	10	15	0



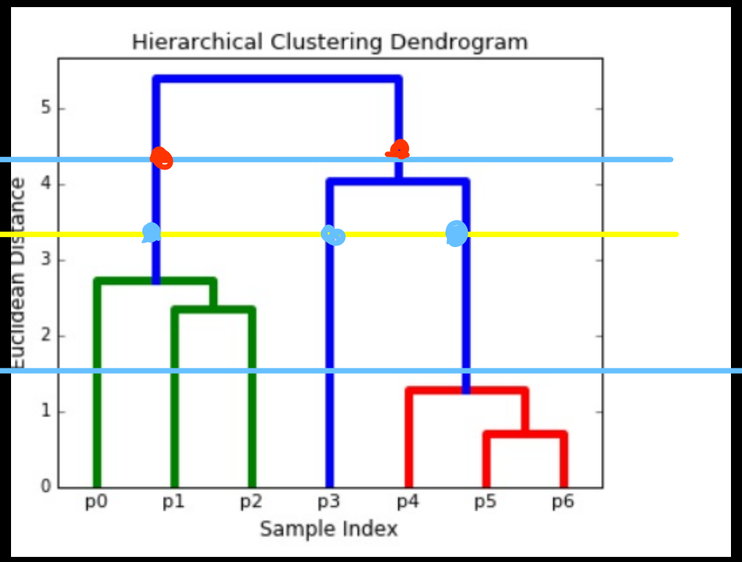
Student ID	Marks
(1,2,4)	20
(3,5)	35

⇒

	(1,2,4)	(3,5)
(1,2,4)	0	15
(3,5)	15	0

→ No of clusters

= No of vertical lines intersected by lines drawn using threshold



→ Dist b/w cluster

1) Single linkage ✓

2) Cross linkage ✓

3) Centroid linkage ✓

4) Ward Linkage ✓

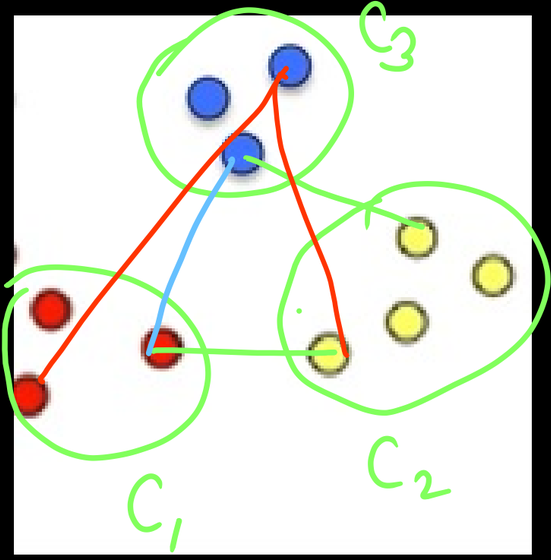
→

✓ Single linkage : Dist. b/w closest point in clusters

$$D(C_1, C_2) = \min_{x_1 \in C_1,\ x_2 \in C_2} D(x_1, x_2)$$

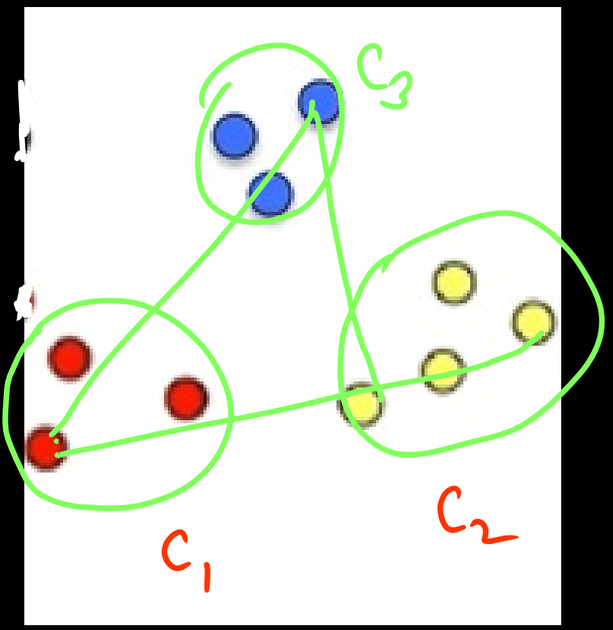
$D(C_1, C_3)$

$D(C_2, C_3)$



Cross linkage: Dist b/w 2 farthest point b/w clusters

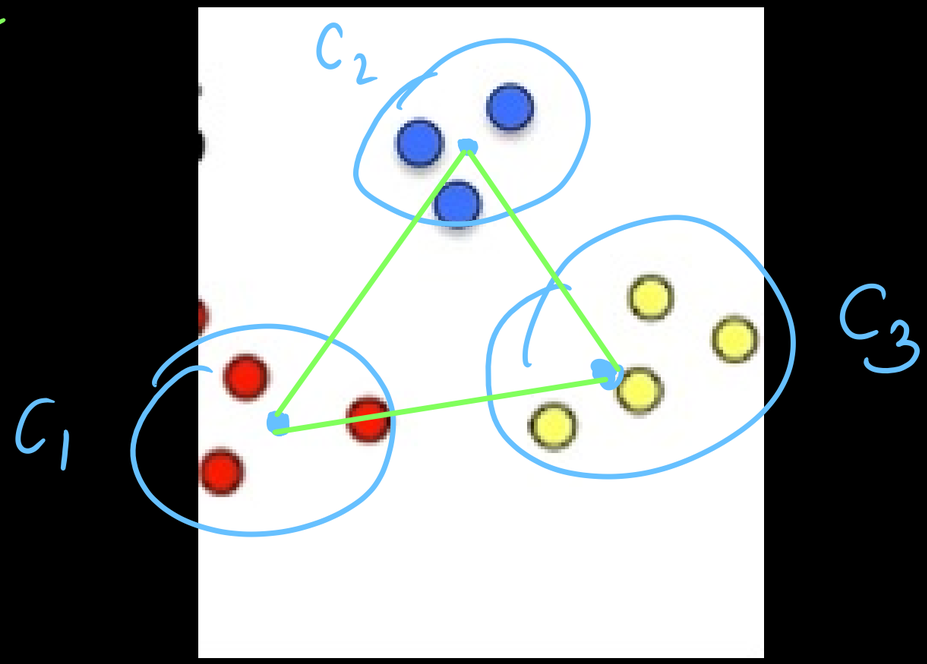
$$D(C_1, C_2) = \max_{x_1 \in C_1,\ x_2 \in C_2} D(x_1, x_2)$$



C1 & C3

→ Centroid linkage

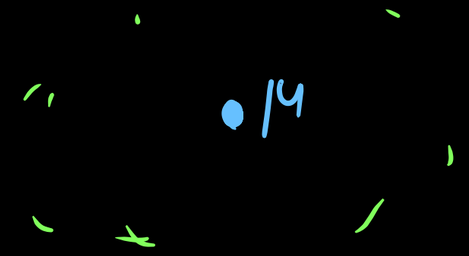
dist (C_1, C_2)

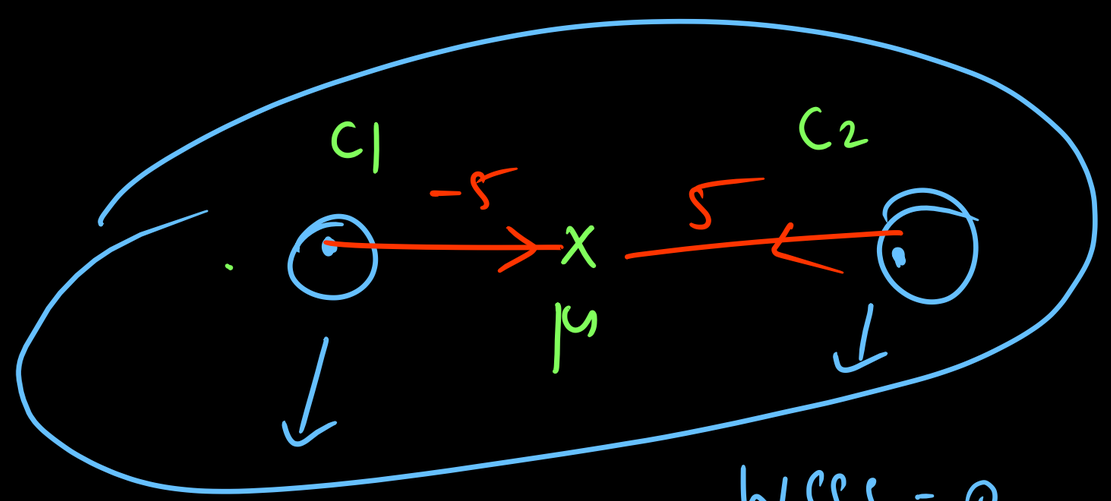


Ward Linkage

WCSS = Within Cluster Sum of Square

$$= \sum_{i \in C} \| x_i - \mu \|^2$$





WARD = WCSS Inc

$= WCSS(C1 \& C2)$

$-[WCSS(C1) + WCSS(C2)]$

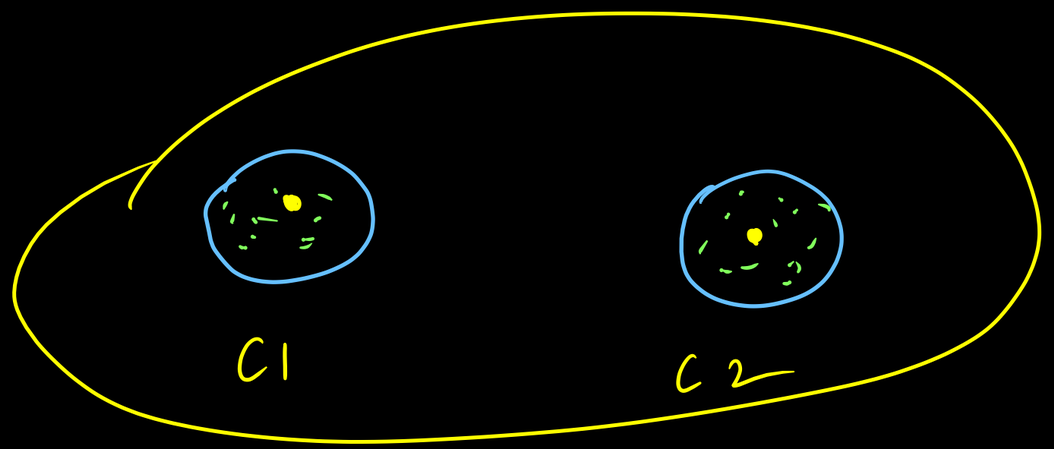
$= \sum_{x_i \in C} \|x_i - \mu\|^2$

$= (-5)^2 + (+5)^2$

→ Wand Linkage

✓ 1) WCSS (C1 & C2)

2) WCSS (C1) + WCSS (C2)



Wand Linkage

WCSS ↑ = WCSS (C1 & C2)

−(WCSS (C1)

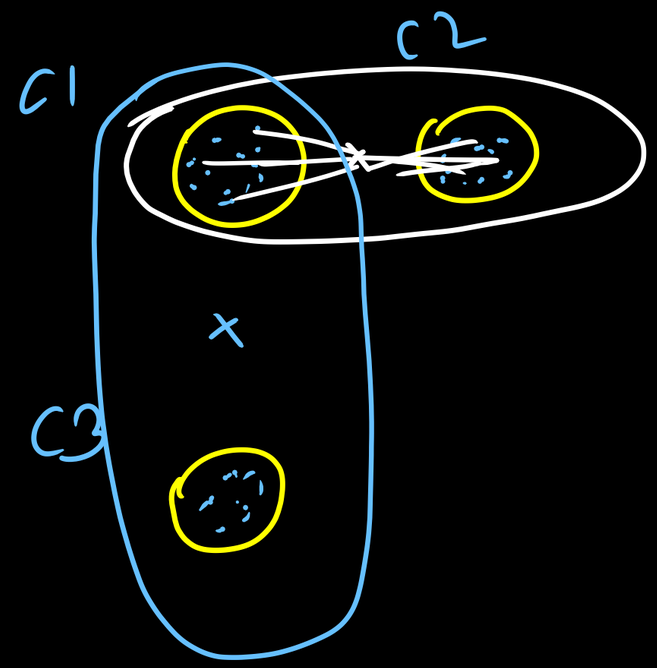
+ WCSS (C2))

$$WCSS(C1, C2) = WCSS(C1 \& C2) - (WCSS(C1) + WCSS(C2))$$

$WCSS\ (C1, C2)$
Inc

$WCSS\ (C2, C3)$
Inc

$WCSS\ (C1, C3)$
Inc



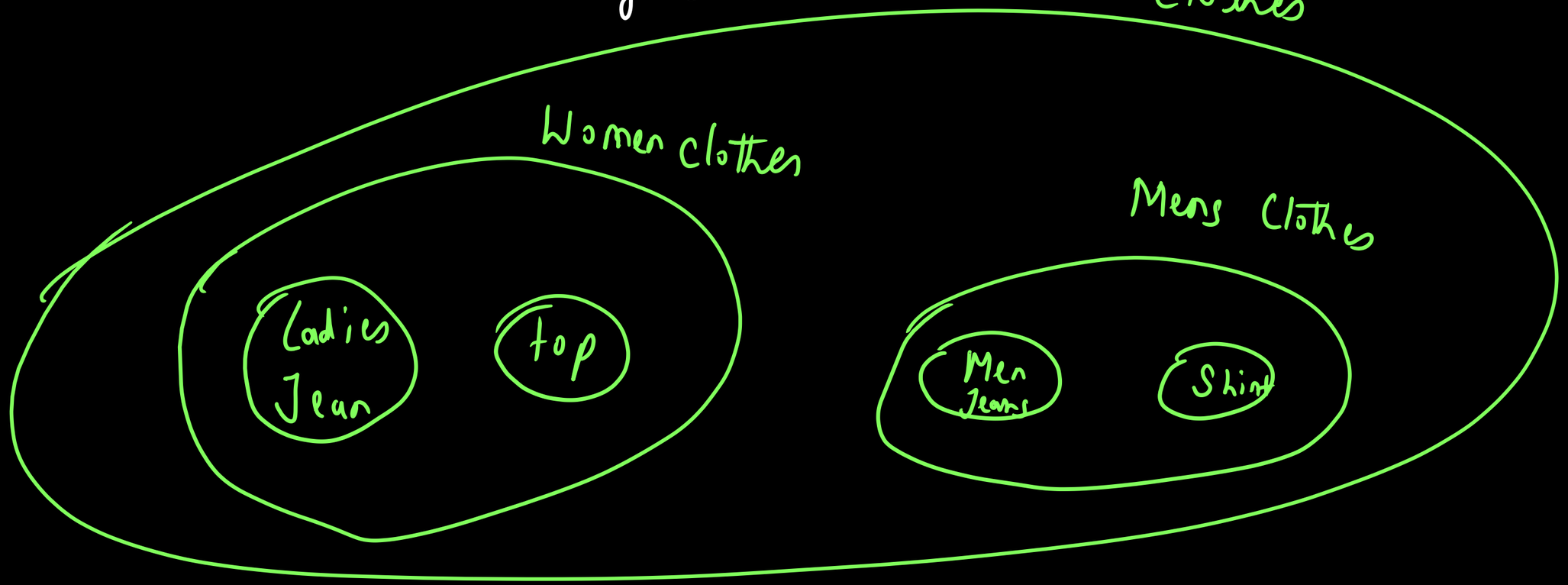
Lowest $WCSS_{Inc}$

Break until 22:31 PM

Advantage

→ No need to specify no of clusters

→ EDA analysis



Disadvantage

↳ Slow in speed

→ Repeat until tot. cluster = 1 → N

→ Calculate the pairwise dist. using Proximity matrix → $O(N^2)$

→ Merge cluster by identifying closest cluster

Time Complexity

↳ $N * O(N^2)$

$= O(N^3)$

↳ Space Complexity → $O(N^2)$

DS/ML

→ ML Breadth ✓

→ ML Depth —

→ ML System Design ✓

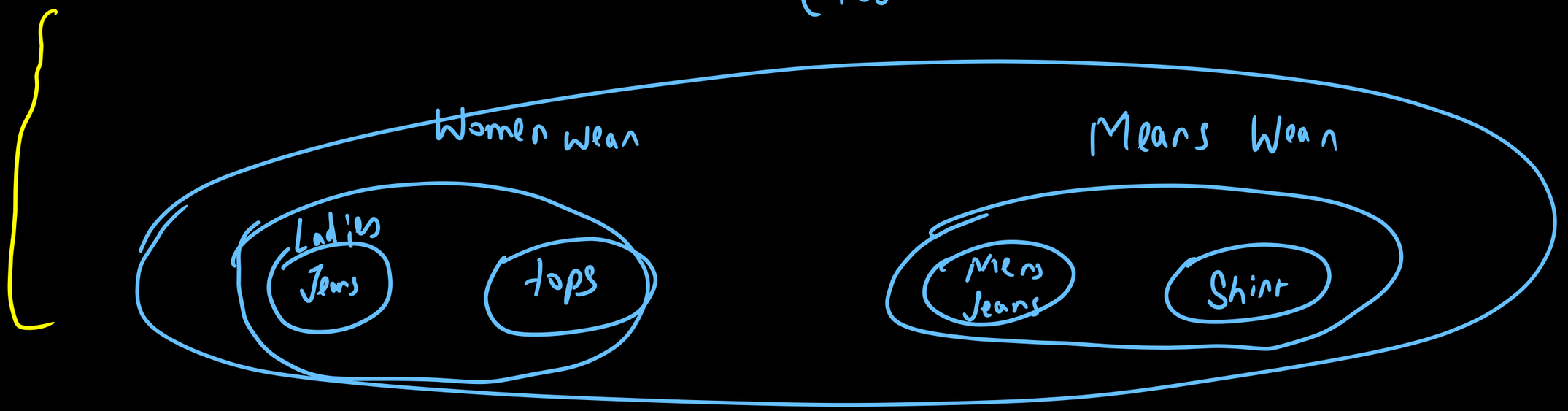
→ DSA + Data Structure & Algo / ML coding

→ HM

Advantage

→ No need to decide no of clusters

→ EDA analysis



→ Disadvantage

↳ Slow

Space Complexity = $O(n^2)$ ← Proximity matrix

n : No of datapoint

Time Complexity = $O(n^3)$

Repeat until tol. cluster = 1 → n

→ Merge cluster → $O(n^2)$

→ Updating Proximity matrix